

Relatório 14 - Redes Neurais (II)

Felipe Fonseca

1. Introdução

Esse relatório foi feito com base em um curso de Deep Learning, abordando desde o funcionamento de um neurônio, até a réplica dele em forma matemática utilizando de algoritmos como Backpropagation, e por fim então, várias formas de criar redes neurais utilizando principalmente a biblioteca do Tensor Flow, aplicando diferentes técnicas para prever diferentes tipos de resultados, sendo eles categóricos e numéricos.

2. Desenvolvimento

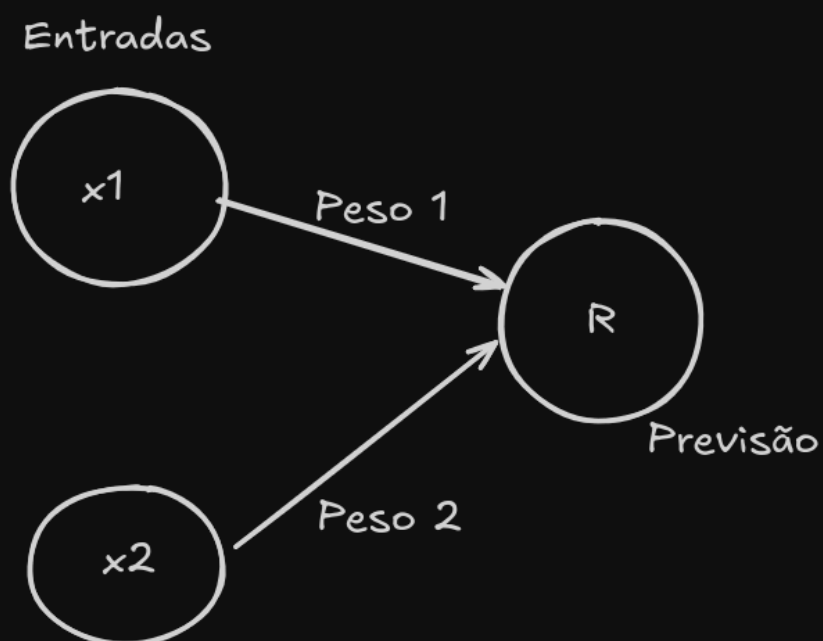
2.1 Teoria

No início do card é ensinado sobre o funcionamento de uma rede neural, tudo em parte teórica, com cálculos e desenhos feitos a mão. Para o primeiro exemplo, o professor simulou um perceptron tendo apenas uma camada de decisão, onde em um exemplo com 2 entradas, sendo ela dois valores binários x_1 e x_2 , e um resultado a ser previsto r , sendo "AND" a operação escolhida.

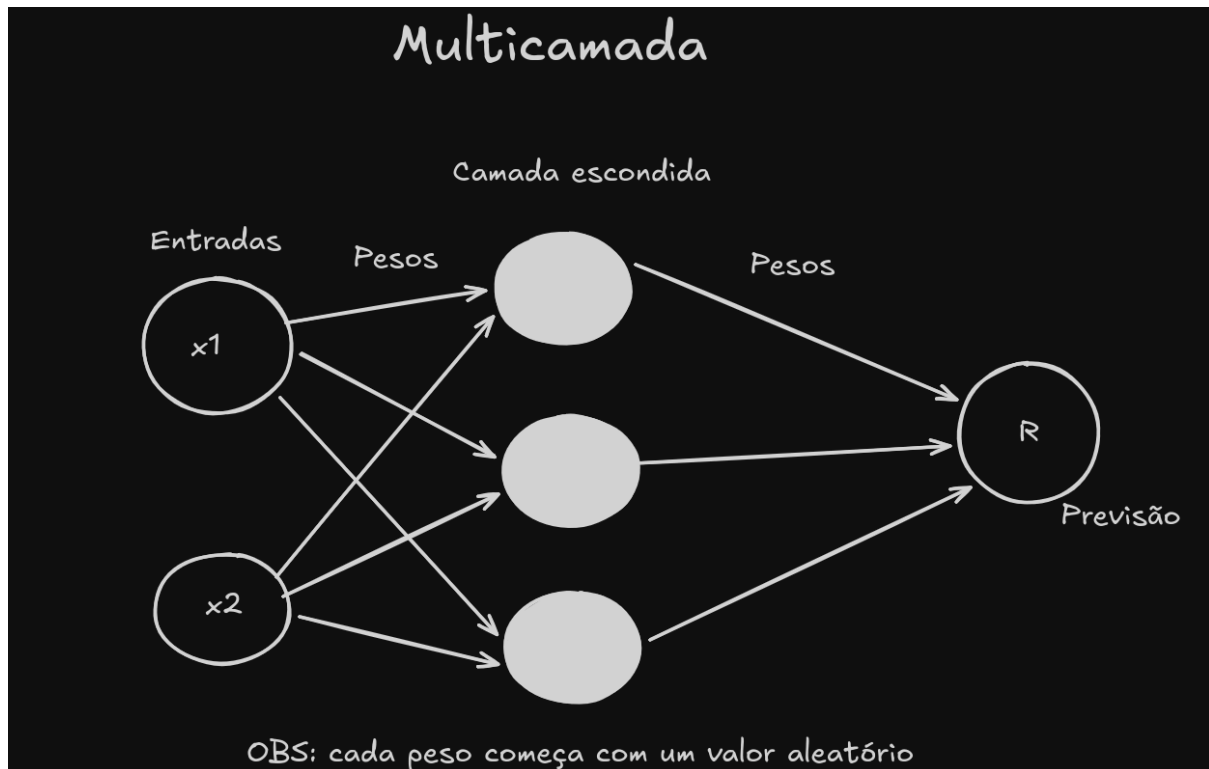
Com base nisso, para cada entrada o perceptron atribui um peso 0 e faz as previsões, e depois um cálculo para saber o quanto de erro ele obteve na previsão. Em busca de aumentar sua precisão, ele atualiza os pesos conforme os resultados anteriores e mais a taxa de aprendizado, atributo escolhido pelo programador.

O perceptron pode ser imaginado dessa forma:

Uma camada



Após esse passo, o instrutor se propôs a apresentar mais termos e fazer mais cálculos para uma rede neural, mas antes, teve de apresentar uma rede neural multicamada, isto é, possui camadas intermediárias entre o resultado e as entradas, assim como na imagem abaixo:



Um dos atributos importantes que foi apresentado é o cálculo do erro, que pode ser calculado de várias formas, sendo a apresentada no vídeo, simplesmente a média absoluta da diferença entre a resposta correta e a previsão.

O próximo passo foi a realização de cálculos como calcular o delta da função de ativação e fazer os cálculos de reajuste de pesos. Esse processo é conhecido como Backpropagation, ou seja, um algoritmo que permite que o modelo ajuste os pesos. Eu refiz todo o processo apresentado no vídeo na mão, e aqui estão os resultados:



Para melhor visualização, junto do relatório no github, também deixarei o PNG e o arquivo do Excalidraw.

Alguns parâmetros que são obrigatórios conhecer:

- Learning rate: taxa de aprendizado, ou seja, o quão rápido o ajuste dos pesos é feito.
- Batch Size: tamanho do lote, ou seja, a quantidade de registros que será enviado por vez para a atualização dos pesos.
- Epochs: épocas, ou seja, quantas vezes devemos fazer a atualização dos pesos.

E para finalizar a parte teórica do card, foi feito uma apresentação das principais funções de ativação que são utilizadas em modelos de ML:

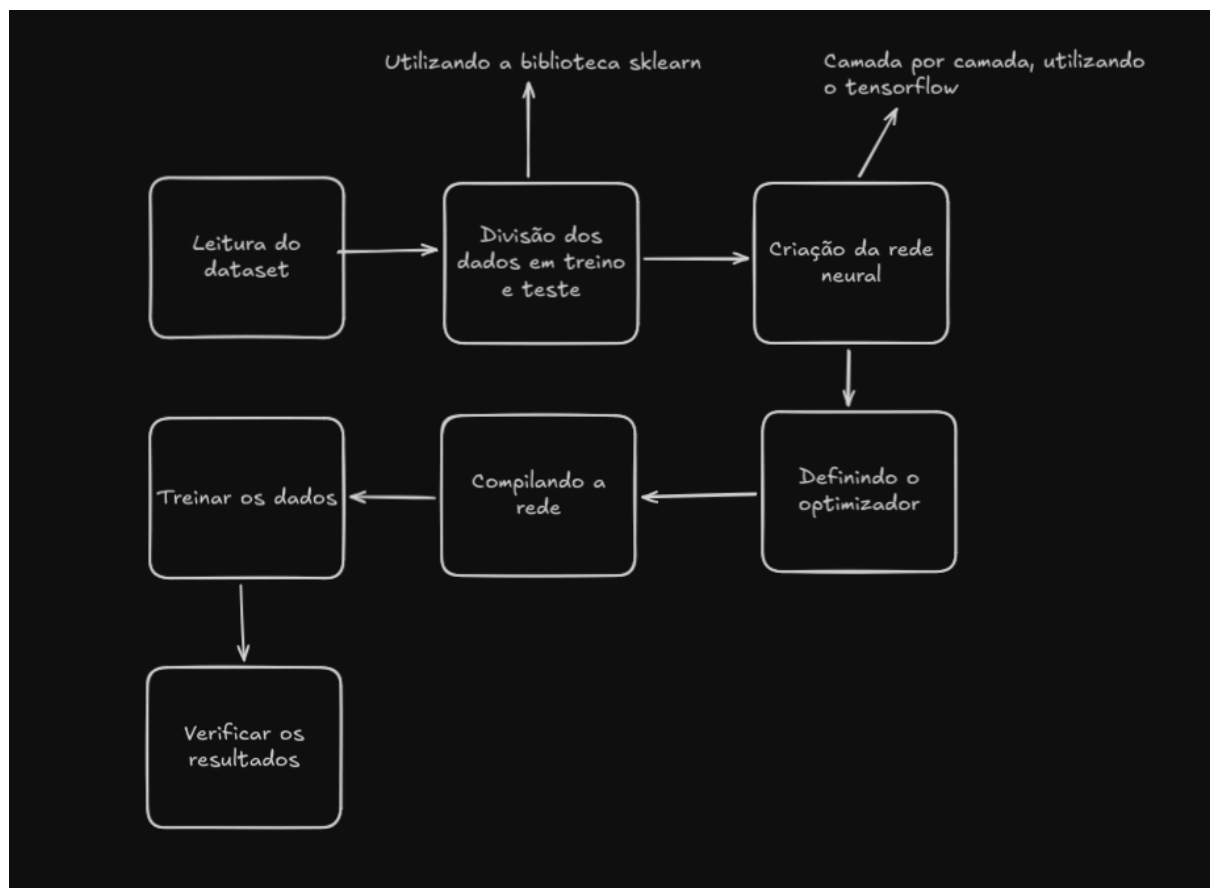
- Step Funcion (função degrau): retorna 0 para valores negativos e 1 para valores positivos.

- Função sigmoide: retorna valores entre 0 e 1.
- Função tangente hiperbólica: retorna valores entre -1 e 1.
- Função relu (a mais utilizada nas camadas ocultas): retorna 0 para valores negativos, e retorna o próprio valor para valores positivos.
- Função softmax (ideal para classificação): retorna uma probabilidade para cada possível resposta.
- Função linear(ideal para regressão): retorna o próprio valor.

2.2 Classificação Binária

O primeiro desafio foi fazer um modelo de previsão para saber se um tumor é maligno ou benigno, ou seja, uma classificação binária.

O dataset utilizado foi o Breast Cancer Wisconsin Diagnostic.



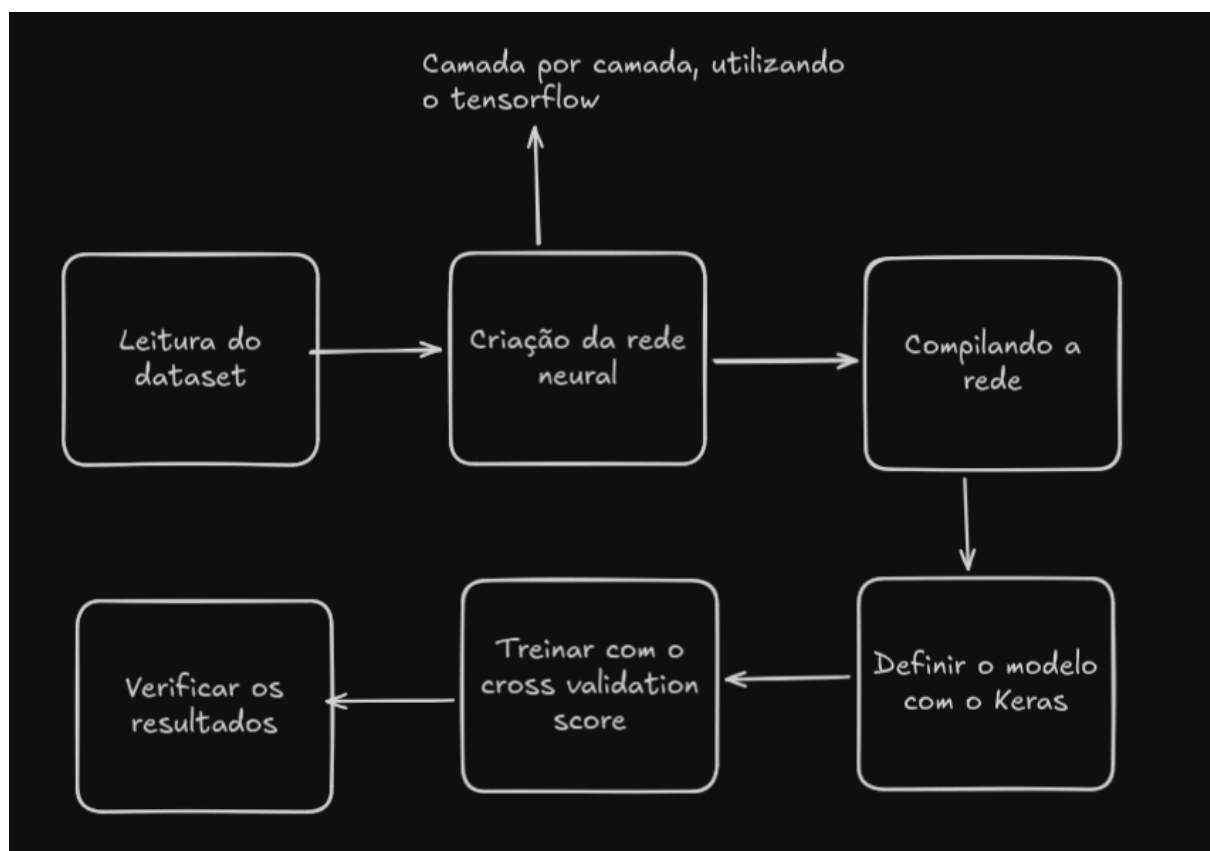
Para fazer a separação dos dados de treino e teste do dataset, foi utilizado o train test split do sklearn, pois seleciona os valores de forma aleatória e é possível definir a porcentagem que será direcionada para treino e teste.

Para a criação da rede neural, foi utilizado o tensor flow, que permite que criemos a rede camada por camada definindo os atributos de cada uma. Utilizamos então uma camada de entrada com os dados, e depois uma

camada densa, utilizando a fórmula $(\text{entrada} + \text{saída}) / 2$ para definir a quantidade de neurônios. A camada de saída utiliza a função sigmóide, para definir um valor entre 0 e 1, ou seja, a porcentagem do tumor ser ou não maligno ou benigno.

Após alguns testes, colocamos uma outra camada densa, porém os resultados pioraram, o que leva à conclusão que para problemas mais simples, como esse de resolução binária, talvez mais camadas levem a uma complexidade desnecessária.

Na segunda parte, fizemos a avaliação cruzada, que é uma forma de treinamento e teste na qual vamos intercalando entre as porções dos dados.



O treinamento com o cross validation foi feito com 10 folds, ou seja, os dados foram separados em 10 partes para fazer a intercalação entre quais dados seriam treinados e quais seriam testados. O resultado final mostra um array com 10 valores, sendo cada valor a probabilidade de acerto para cada fold.

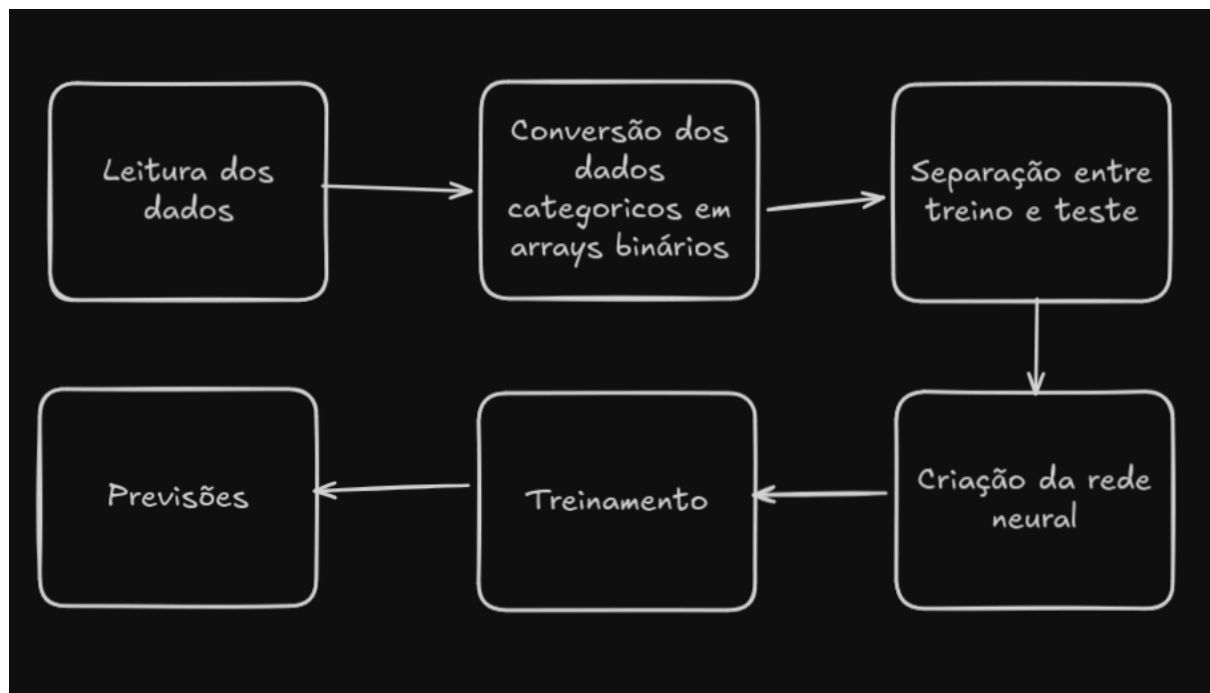
Para a verificação dos melhores parâmetros para o treinamento, utilizamos do Grid Search CV do sklearn, que consegue realizar vários treinamentos intercalando entre parâmetros que passamos para verificar qual a melhor combinação.

Após isso apenas foi ensinado a forma de fazer a previsão de um único novo registro utilizando a função predict, e como salvar o modelo treinado e carregá-lo novamente.

2.3 Classificação Multiclasse

Para o próximo teste, agora um caso em que a resposta não é binária, mas sim uma classificação, ou seja, uma resposta categórica.

O dataset utilizado foi o dataset Iris, um dos mais conhecidos no ML.



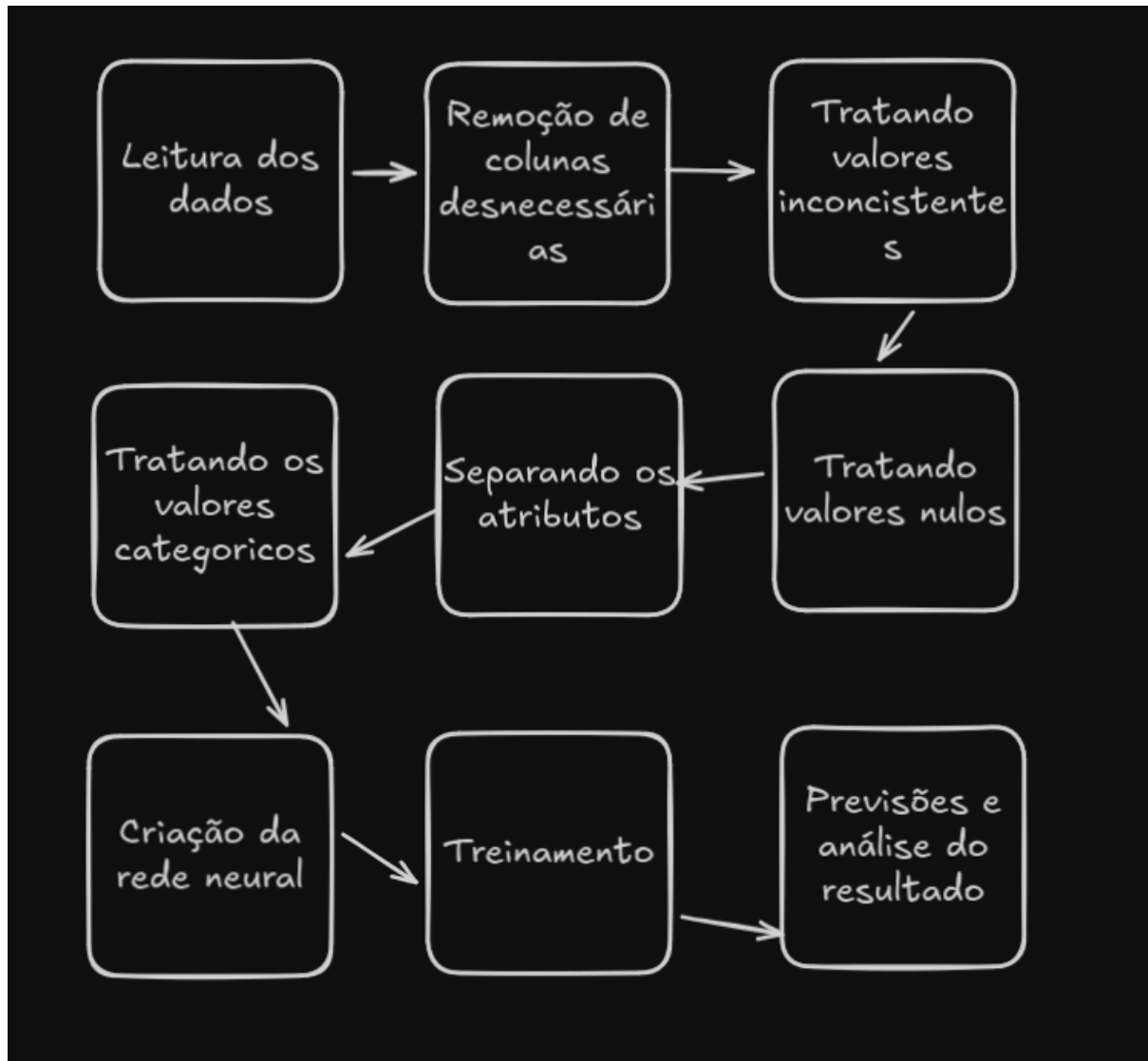
Nesse caso, os 2 principais fatores de atenção foram, primeiramente, a conversão das respostas. A coluna que continha as respostas com o tipo da flor, alternando entre 3 possíveis respostas, teve que ser convertido para um valor numérico, e a melhor forma de fazer isso no machine learning, é criando um array para cada resposta, onde mostramos então a probabilidade da flor ser de cada uma das 3 categorias.

O segundo ponto de atenção foi na função de ativação. Para casos como esse, a função softmax é perfeita, pois ela retorna exatamente isso, uma probabilidade para cada uma das possíveis respostas.

Depois também foi feito uma avaliação cruzada nesse mesmo dataset para ver os resultados.

2.4 Regressão

O terceiro desafio foi fazer uma rede neural de regressão para prever o valor de venda de um carro. Esse foi de longe o mais complexo pela quantidade de tratamentos que deveriam ser feitos antes do treinamento.



Nessa parte, o principal problema foi cuidar dos dados, pois haviam muitos atributos inconsistentes, desnecessários e nulos. Utilizamos de funções para verificar os valores nulos, removemos todas as colunas que não eram necessárias para fazer previsões, e acabando removendo também todas as linhas com valores nulos ou inconsistentes, apesar de não ser o recomendado.

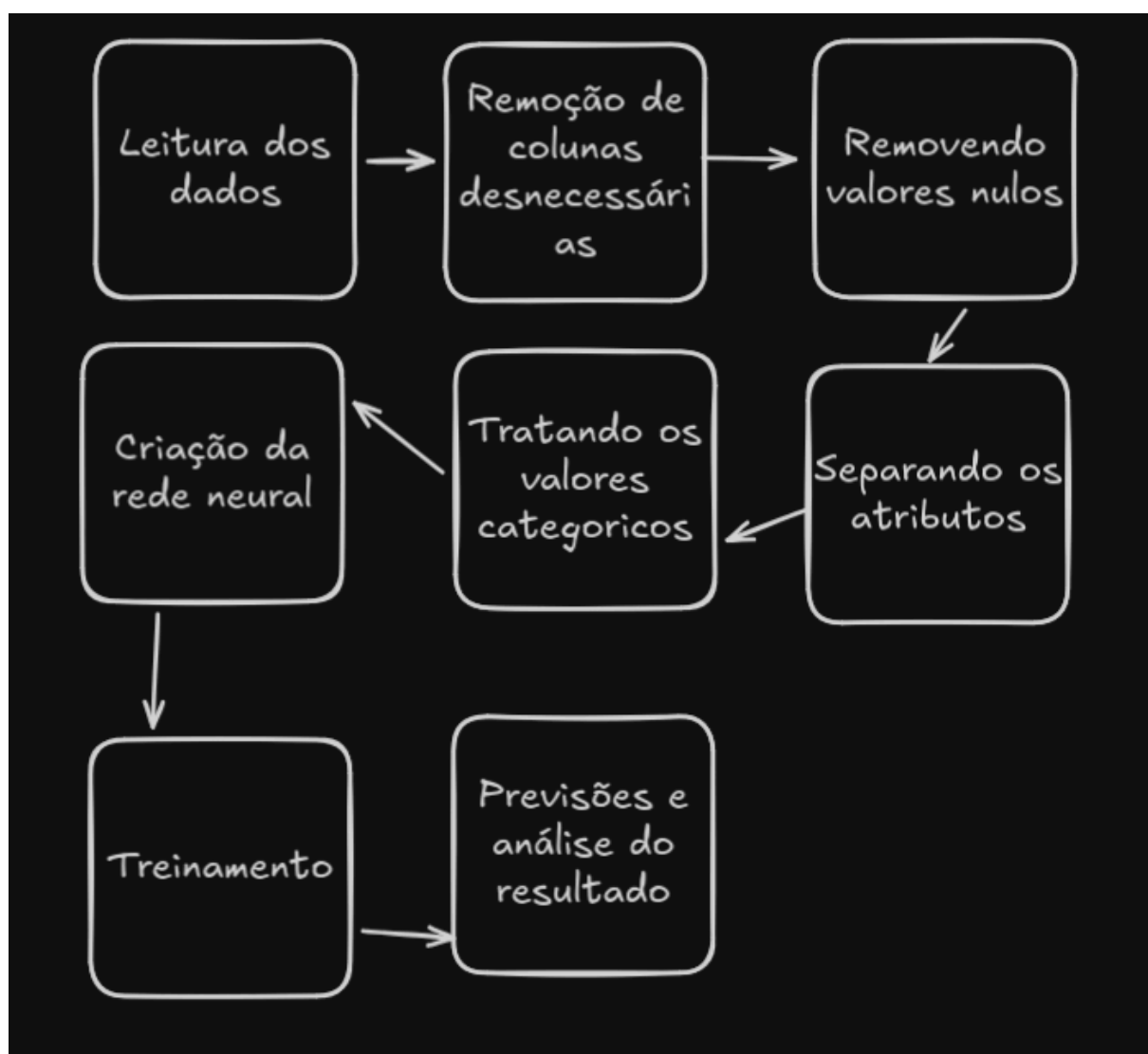
Depois utilizamos uma função chamada Column Transfer junto com o One Hot Encoder para transformar os valores categóricos em arrays binários igual já havíamos feito anteriormente.

Depois criamos a rede neural com 2 camadas densas e fizemos o treinamento, utilizando para a camada de saída a função Linear, que retorna o próprio valor que chegar. Essa função é a recomendada para casos de regressão.

Após isso, recriamos o código de forma simples para fazer o treinamento com a validação cruzada e obter os resultados.

2.5 Regressão com múltiplas saídas

Para o último caso, utilizamos de um dataset de videogames para fazer uma regressão para prever 3 resultados, ou seja, vendas em 3 regiões diferentes.



Neste caso, para agilizar, removemos todos os valores nulos, o que foi metade do dataset, definitivamente não o recomendado. Após isso, fizemos novamente a separação dos atributos e transformamos os valores categóricos em valores numéricos, utilizando o One Hot Encoder. A rede neural foi feita

com 3 camadas de saída, cada uma com 1 valor, isso porque os valores não são necessariamente a mesma coisa, então é uma forma melhor do que colocar 3 resultados em uma só camada de saída.

2.6 Prática

Na prática eu utilizei o dataset de casas para vender. Inicialmente eu tentei utilizar um dataset com dados de aplicativos em lojas de celulares, porém não deu muito certo pela inconsistência dos dados, além de que haviam poucas colunas para serem avaliadas, então eu fiz a troca do dataset.

Por conta dos fatos dos valores já estarem tratados em sua maioria, sem valores nulos e tudo preenchido, achei que seria tranquilo, mas eu me enganei. Fiz a validação cruzada utilizando de um modelo com 2 camadas, obviamente utilizando regressão, porém no final do treinamento a margem de erro estava em mais de 150 mil. Foi então que pensei em padronizar os dados com o MinMax Scaler, e fiz isso para todos os valores. Além disso, tinha um valor despercebido ali no meio, que era o ZipCode, que eu havia esquecido de tratar. O que eu fiz foi transformá-lo em várias colunas com o OneHotEncoder.

Ao final, o modelo ainda teve um erro muito grande, cerca de 80 mil, porém quase metade a menos do que anteriormente.

3. Conclusão

As redes neurais são algoritmos muito úteis para diferentes problemas, tanto de classificações quanto para regressões. A arquitetura em várias camadas da rede neural é algo crucial para seu funcionamento, porém se complexa demais, pode dificultar um problema que é simples. No geral, foi o card mais bem aplicado em conceitos de machine learning, mostrando tudo como as redes neurais realmente funcionam, clareando todo o entendimento sobre o funcionamento de IAs.

4. Referências

Vídeos do Card: Deep Learning com Python de A a Z, sessões 2 à 7;